

PLANTELISTAS

Manual Técnico Oficial - V1

Arquitetura modular, regras de negocio, ciberseguranca, autonomia operacional do Codex e metodologia de entrega

Documentacao v1.1 | 26/08/2026

V1 funcionalmente congelada

1. Objetivo e governanca

Este manual e a referencia humana oficial da V1 do Plantel Listas. O projeto deve permanecer compreensivel mesmo quando houver troca de conta, chat ou sessao do Codex. A documentacao versionada, o codigo, as migrations e os relatorios aprovados sao a fonte de verdade.

Principio operacional: o Codex executa o trabalho tecnico da fase; o responsavel humano revisa, decide regras, aprova merge/deploy e libera a fase seguinte.

2. Visao do produto

O Plantel Listas transforma o acervo existente no Google Drive em uma experiencia propria, autenticada, pesquisavel e profissional para alunos, professores e administradores.

- Drive armazena PDFs e videos.
- MySQL guarda usuarios, permissoes, metadados, analytics, auditoria e estado de sincronizacao.
- React/Vite entrega a interface.
- Node.js/Express concentra regra de negocio, autorizacao, banco e integracoes.

3. Escopo congelado da V1

- Cadastro/login/logout de aluno.
- Login de professor/admin.
- Recuperacao de senha por email.
- Roles aluno, professor e admin.
- Categorias/subcategorias, busca e filtros.
- PDFs e videos: visualizar/assistir/baixar.
- Professor: upload, edicao, movimentacao, substituicao e lixeira em areas autorizadas.
- Admin: usuarios, roles, permissoes, categorias, lixeira, restauracao, exclusao definitiva, analytics e auditoria.
- Google Drive API + indexacao inicial do acervo.
- Analytics exclusivo do admin.
- Responsividade, testes, ciberseguranca e deploy Hostinger.

4. Roadmap fora da V1

4.1 V2

Banco de Questoes.

4.2 V2.x

- listas personalizadas;
- salvar questoes;
- historico e questoes resolvidas;
- geracao de simulados;
- estatisticas de desempenho.

4.3 V3

- planos/premium;
- pagamentos PIX/cartao;
- beneficios por plano;
- gestao de assinatura;
- webhooks;
- controle financeiro.

O Codex nao deve implementar nem preparar complexidade desnecessaria desses modulos durante a V1.

5. Stack e arquitetura

Camada	Definicao oficial
Frontend	React + Vite + JavaScript
Backend	Node.js + Express + CommonJS
Arquitetura backend	Monolito modular por dominio
Banco	MySQL / InnoDB / utf8mb4
Local	Laragon + MySQL + MySQL Workbench
Arquivos	Google Drive via Drive API
Producao	Hostinger Business Web Hosting

5.1 Monolito modular por dominio

```
backend/src/  
  modules/  
    autenticacao/  
    usuarios/  
    materiais/  
    categorias/  
    permissoes/  
    analytics/  
    auditoria/  
  shared/  
    config/  
    database/  
    errors/  
    middlewares/  
    providers/  
    utils/  
  server.js
```

Cada modulo possui seus routes/controllers/services/repositories/validators conforme necessidade. Nao criar pastas globais misturando todos os dominios. Nao criar microservicos.

5.2 Separacao interna

- Route: endpoint, middlewares e controller.
- Controller: camada HTTP, sem regra de negocio.
- Service: regra de negocio e coordenacao.
- Repository: MySQL com SQL parametrizado.
- Validator: formato de entrada.
- Shared: apenas recursos realmente compartilhados.
- Provider: integracoes externas, incluindo Google Drive.

6. Autonomia operacional do Codex

O Codex deve fazer diretamente o trabalho tecnico que estiver autorizado e ao alcance do ambiente. Nao deve transformar execucao em tutorial para o responsavel humano.

- criar estrutura e arquivos;
- inicializar/configurar packages;
- instalar dependencias da fase;
- criar banco MySQL local quando houver acesso;
- criar e executar migrations;
- rodar backend/frontend;
- executar testes e auditoria de dependencias;
- corrigir erros;
- criar branch/commits e enviar branch quando autenticacao Git estiver disponivel;
- gerar relatorio e parar.

Intervencao humana somente quando inevitavel: segredo nao disponivel, OAuth no navegador, acesso a painel externo, decisao de negocio, merge em main, deploy ou operacao destrutiva. Nunca pedir senha da conta Google.

6.1 Dependencias e banco

Dependencias sao instaladas progressivamente por fase. O Codex deve justificar cada uma e manter lockfile. O banco local padrao e `plante\_listas`; se o MySQL estiver acessivel, o Codex deve cria-lo e testa-lo sem pedir que o usuario faca manualmente. As tabelas surgem conforme os modulos/fases, evitando modelagem prematura.

7. Padrao de codigo

- CommonJS (`require` / `module.exports`).
- Funcoes tradicionais nomeadas.
- Nao usar arrow functions em codigo de primeira parte, inclusive testes, salvo incompatibilidade tecnica aprovada.
- Variaveis e funcoes de negocio preferencialmente em portugues.
- Nomes explicitos, sem abreviaco es obscuras.
- Codigo simples e previsivel; evitar atalhos que prejudiquem a leitura.
- `const` por padrao; `let` apenas com reatribuicao.
- Controller nao contem regra de negocio.
- Repository nao decide regra de negocio.

```
function verificarPermissaoDeAdmin(req, res, next) {  
  const usuario = req.usuario;  
  const papelDoUsuario = usuario.papel;  
  
  if (papelDoUsuario !== "admin") {  
    throw new AppError("Usuario sem permissao", 403);  
  }  
  
  next();  
}
```

8. Perfis e permissoes

Acao	Aluno	Professor	Admin
Cadastro proprio	Sim	Nao como professor	Nao como admin
Acessar/baixar	Sim	Sim	Sim
Upload	Nao	Area autorizada	Sim
Editar/mover/substituir	Nao	Area autorizada	Sim
Enviar para lixeira	Nao	Area autorizada	Sim
Restaurar	Nao	Nao	Sim
Excluir definitivamente	Nao	Nao	Sim
Usuarios/roles	Nao	Nao	Sim
Permissoes	Nao	Nao	Sim
Analytics/relatorios	Nao	Nao	Sim
Auditoria geral	Nao	Nao	Sim

Regra absoluta: toda permissao deve ser revalidada no backend, mesmo quando o frontend esconder a acao.

9. Autenticacao e recuperacao de senha

- Cadastro publico cria somente aluno.
- Email unico.
- Professor/admin somente por acao administrativa.
- Senha armazenada com hash seguro.
- Preferencia: sessao por token opaco forte, hash no banco e cookie HttpOnly/Secure/SameSite.
- Nao armazenar segredo de sessao em localStorage.
- Protecao CSRF nas operacoes de estado.
- Rate limit em autenticacao e recuperacao.

Recuperacao: resposta neutra, token forte temporario e de uso unico, hash do token no banco, redefinicao de senha e invalidacao do token/sesoes conforme politica aprovada.

10. Google Drive

Pasta raiz do acervo:

10Kokm2f3Ipe0FuzIJvJDc4HpHZpBo0IX

Raiz confirmada: LISTAS e PROVAS ANTIGAS. A arvore nao deve ser hard-coded.

- OAuth 2.0 server-side.
- Senha Google nunca entra no sistema.
- Credenciais e refresh token somente no backend/ambiente seguro.
- Provider isolado em shared/providers.

- Avaliar menor escopo possivel.
- `drive.file` deve ser avaliado primeiro, mas o acervo legado pode exigir outra abordagem.
- Escopos amplos do Drive sao restritos e podem exigir verificacao/avaliacao adicional.
- Nao considerar OAuth em status Testing uma solucao definitiva de producao para escopos nao basicos: refresh tokens podem expirar em 7 dias.

10.1 Indexacao e consistencia

- Indexacao inicial idempotente.
- Referencia por `drive\_file\_id`.
- Registro das sincronizacoes.
- Reindexacao administrativa.
- Operacoes normais pelo Plantel Listas apos go-live.
- Falhas parciais entre Drive e MySQL devem ser tratadas/compensadas.

10.2 Upload

- Autenticacao e autorizacao por area.
- MIME/extensao/tamanho validados.
- Nome seguro.
- Temporarios removidos.
- Nenhum arquivo executado.
- Salvar referencia no banco somente apos operacao coerente no Drive.
- Testar videos reais na Hostinger antes do go-live.

11. Analytics e auditoria

Analytics e exclusivo do admin. Metricas: alunos ativos/dia, acessos, visualizacoes, downloads, buscas/termos, materiais e categorias mais acessados, horarios e evolucao semanal/mensal.

Auditoria registra quem, o que, quando e alvo para alteracoes administrativas e operacoes sensiveis. Nunca registrar senha, token ou segredo OAuth.

12. Ciberseguranca e privacidade

- Backend e autoridade final.
- SQL parametrizado.
- Protecao contra IDOR, XSS, CSRF, brute force, enumeracao de email e mass assignment.
- Upload seguro e MIME spoofing.
- Headers HTTP e cookies seguros.
- CORS restritivo conforme arquitetura.
- Segredos somente em ambiente seguro.
- Logs sem segredos/dados pessoais desnecessarios.
- Dependencias auditadas.
- Minimizacao de dados pessoais e controle de acesso a analytics/relatorios.

13. Git e fluxo de aprovacao

- `main` = estado estavel/aprovado.
- Uma branch por fase, exemplo `fase/01-fundacao`.
- Codex pode criar branch, commits e push da branch.
- Merge em main somente com aprovacao humana.
- Sem force push, reescrita de historico ou alteracao de remote.
- Cada fase gera `docs/relatorios/RELATORIO\_FASE\_XX.md`.

14. Hostinger

O ambiente de producao definido e Hostinger Business Web Hosting. A documentacao oficial atual da Hostinger informa suporte a Node.js/Express nesse plano, deploy via GitHub/upload e configuracao de build, start e variaveis de ambiente no hPanel.

A mesma documentacao informa que, para Node.js Web Apps em Business/Cloud, npm e executado pelo fluxo de deploy e nao manualmente via SSH. Por isso, a estrategia de migrations de producao deve ser validada dentro do fluxo suportado antes do go-live, sem presumir shell persistente.

- validar versao Node;
- root directory;
- build command;
- start command;
- env vars;
- migrations em producao;
- limites praticos de upload/video;
- smoke test antes de apontar trafego.

15. Fases de implementacao

Fase	Entrega
0	Documentacao e preparacao.
1	Estrutura, monolito modular, banco local/migrations-base e seguranca.
2	Cadastro, login, sessoes, roles e recuperacao de senha.
3	Categorias, disciplinas, concursos, metadados e permissoes.
4	Drive + indexacao/importacao inicial.
5	Consulta, busca, filtros, PDF/video/download.
6	Upload, edicao, movimentacao, substituicao e lixeira.
7	Admin, usuarios, permissoes, analytics e auditoria.
8	Acabamento frontend, responsividade e UX.
9	QA, seguranca, carga, deploy e smoke.

Fluxo: ANALISAR -> PLANEJAR -> EXECUTAR -> TESTAR -> CORRIGIR -> RELATAR -> PARAR.

16. Relatorio obrigatorio

- resumo e branch/commit;
- arquivos e estrutura;
- dependencias/versoes/justificativas;
- migrations, banco, indices e constraints;
- endpoints/regras/permissoes;
- seguranca;
- testes e resultados;
- erros encontrados/corrigidos/abertos;
- riscos e pendencias;
- divergencias;
- sugestoes de backlog sem implementar;
- estado PRONTA PARA VALIDACAO ou NAO PRONTA;
- proxima fase sugerida sem iniciar.

17. Referencias tecnicas verificadas em 26/08/2026

Hostinger - Node.js hosting options: <https://www.hostinger.com/support/node-js-hosting-options-at-hostinger/>

Hostinger - Deploy Node.js Web App: <https://www.hostinger.com/support/how-to-deploy-a-nodejs-website-in-hostinger/>

Hostinger - Redeploy/build/start/env: <https://www.hostinger.com/support/how-to-redeploy-a-node-js-application/>

Google Drive - escolher escopos OAuth: <https://developers.google.com/workspace/drive/api/guides/api-specific-auth>

Google OAuth 2.0 server-side: <https://developers.google.com/identity/protocols/oauth2/web-server>

Google OAuth 2.0 - refresh token/testing: <https://developers.google.com/identity/protocols/oauth2>

18. Estado para inicio

Com esta documentacao v1.1, a Fase 1 pode ser iniciada. Antes disso, substituir a documentacao anterior no repositorio. O Prompt Inicial da Fase 1 instrui o Codex a trabalhar em branch propria, executar a fundacao tecnica, testar, gerar relatorio e parar sem merge/deploy.